SYSTEM DESIGN TRAINING PROGRAM

Day 2

SOLID Principles & Maintainable Code

Case Study: RetailOrderHub — refactoring OrderManager

Language: Java **Platform:** GitHub

Today's Agenda

Four hours: principles, a live demo, two labs, a break, and a final lab

01	**Recap & Bridge from Day 1**	From yesterday's findings to today's fixes
02	**SOLID Principles Overview**	Five principles, one goal: code that's easy to change
03	**SRP, OCP, LSP, ISP, DIP + Demo**	Each principle explained and applied — including a live PaymentService extraction demo
04	**Lab 1 + Lab 2**	SRP→DIP refactor, then the OCP exercise — right after the principles that drive them
05	**Version Control, Break + Lab 3**	Git via GitHub, a 20–30 minute break, then your own branch→PR→merge workflow

Yesterday

We Found the Problems

From Lab 3's findings log

- God Object: processOrder() does validation, inventory, payment, and persistence in one method
- Duplicated Code: validateCustomer/validateItems repeat inline logic
- Primitive Obsession: payment method handled via if/else on raw strings

Today, We Fix the Structure

Every finding yesterday traces back to one root cause: OrderManager has too many reasons to change

SOLID gives us five concrete principles to break that apart safely

By the end of today, OrderManager becomes OrderService, PaymentService, and InventoryService — each with one job

This isn't refactoring for its own sake — it directly fixes the God Object and Duplicated Code findings from Lab 3

SOLID Principles — Overview

Five principles, one goal: code that's easy to change safely

S

Single Responsibility

A class should have one, and only one, reason to change

O

Open-Closed

Open for extension, closed for modification

L

Liskov Substitution

Subtypes must be substitutable for their base types

I

Interface Segregation

Many small, client-specific interfaces beat one large one

D

Dependency Inversion

Depend on abstractions, not concretions

Why SOLID exists: reducing coupling, easing change, improving testability — the antidote to yesterday's God Object.

Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change. Each responsibility is an axis of change — bundle several together, and a change driven by one actor risks breaking something another actor depends on.

Symptom 1: Accidental Duplication

When multiple responsibilities share a class, logic that looks reusable often isn't — one actor's change accidentally breaks another actor's behavior, because both paths were sharing code that only looked common.

Symptom 2: Merge Conflicts

Two developers, each working on a different responsibility, keep editing the same file for unrelated reasons — constant merge conflicts are a strong signal that a class is doing more than one job.

In RetailOrderHub: OrderManager.processOrder() answers to three different actors — the fulfillment team (inventory), finance (payment), and customer service (order status) — all coupled into one class.

Applying SRP to RetailOrderHub

From one overloaded class to three focused collaborators

BEFORE

OrderManager

- processOrder()
- validateCustomer()
- validateItems()
- getInventoryQuantity()

One class, four unrelated jobs

AFTER

OrderService	InventoryService	PaymentService
createOrder()	checkStock() / reserve()	charge()

Three classes, one job each — each with its own reason to change

Fixes from Lab 3: God Object — resolved (three classes, three actors). Duplicated Code — resolved (validation lives in exactly one place per concern, not scattered across two unused helper methods).

Live Demo: Extracting PaymentService

Trainer-led walkthrough • ~10–12 minutes • SRP applied to OrderManager

Live Demo: Extracting PaymentService (SRP)

- Identify the seam: the “Process payment” block — clean input/output, no DB or item-list coupling
- Create PaymentService.java with one method: charge(paymentMethod, amount)
- Wire it into OrderManager as a constructor dependency, replacing the if/else block with one call
- Rebuild, restart, and replay the same test order — prove behavior didn't change
- Hand-off: Lab 1 finishes the rest (InventoryService, OrderService) through DIP

Open-Closed Principle (OCP)

A software artifact should be open for extension, but closed for modification. Behavior should be extendable without having to modify the artifact itself. If a simple new requirement forces changes across existing code, that's a design failure.

A Thought Experiment

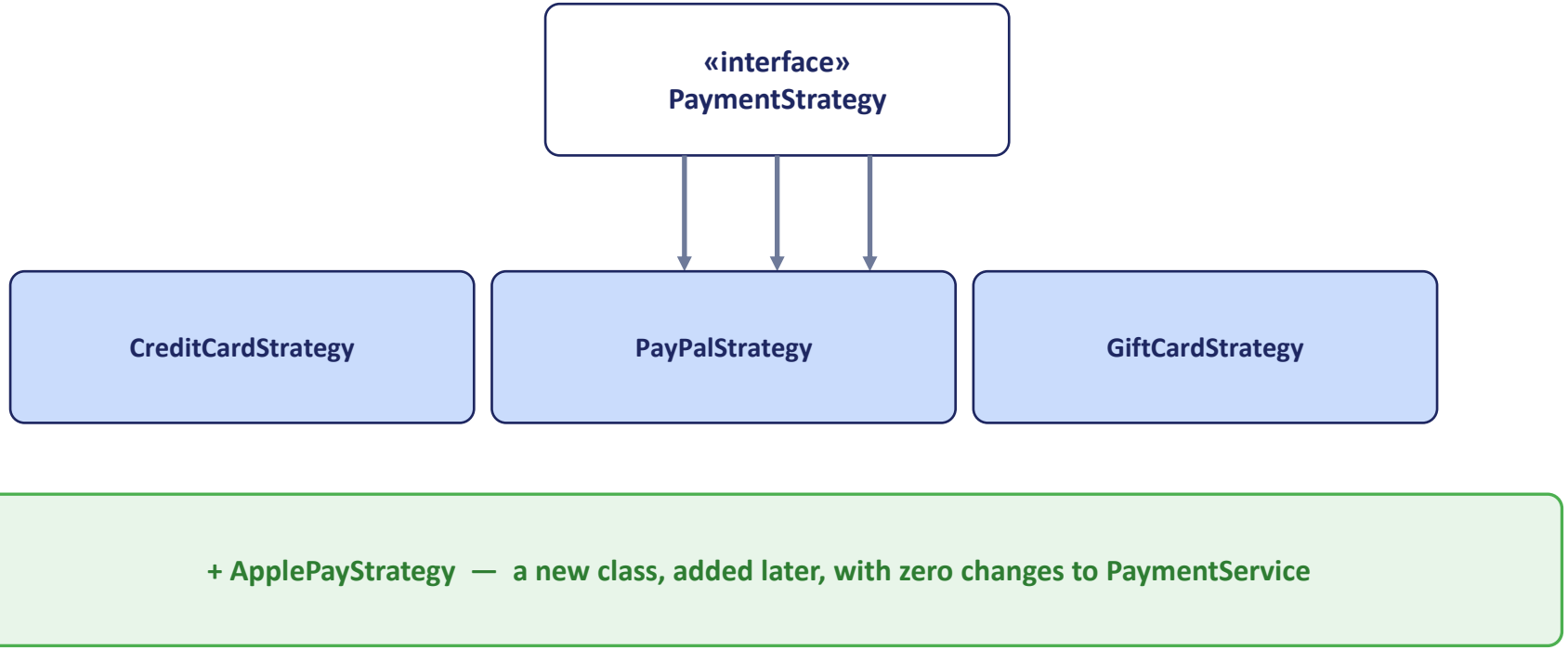
A financial summary page shows scrollable data with negative numbers in red. Stakeholders now want the same data printed on a black-and-white printer, with negative numbers in parentheses instead. How much old code has to change? A good architecture reduces that to the barest minimum — ideally zero.

In RetailOrderHub

- Today: adding a new payment method means editing an if/else chain inside PaymentService — modifying existing, tested code
- Goal: adding Apple Pay or a BNPL provider should mean writing one new class, touching nothing that already works
- This is exactly what the OCP exercise asks you to build

Applying OCP: The Strategy Pattern

Extending payment methods without touching existing code



PaymentService now depends on the PaymentStrategy interface, not on if/else logic. New payment methods are new classes — PaymentService's code never changes again.

Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types. If code works correctly with a base type, it must keep working correctly when any subtype is used in its place — no surprises, no broken assumptions.

Applied to Every PaymentStrategy

- PaymentService should be able to call charge(amount) on any PaymentStrategy implementation and get consistent, predictable behavior
- If GiftCardStrategy silently does nothing when the balance is insufficient, while CreditCardStrategy throws an exception, that's an LSP violation — callers can't treat them interchangeably
- Every strategy must honor the same contract: succeed, or fail loudly and consistently

Rule of thumb: if you find yourself checking "what type of strategy is this" before calling a method, LSP is being violated somewhere.

Interface Segregation Principle (ISP)

Many small, client-specific interfaces beat one large one

BEFORE — One Fat Interface

OrderRepository

- findById()
- save()
- delete()
- generateInvoiceReport()
- exportToAccountingSystem()
- notifyWarehouse()

A reporting client is forced to depend on warehouse and accounting methods it never calls

AFTER — Segregated by Client

OrderRepository

findById() / save() / delete()

InvoiceReporter

generateInvoiceReport()

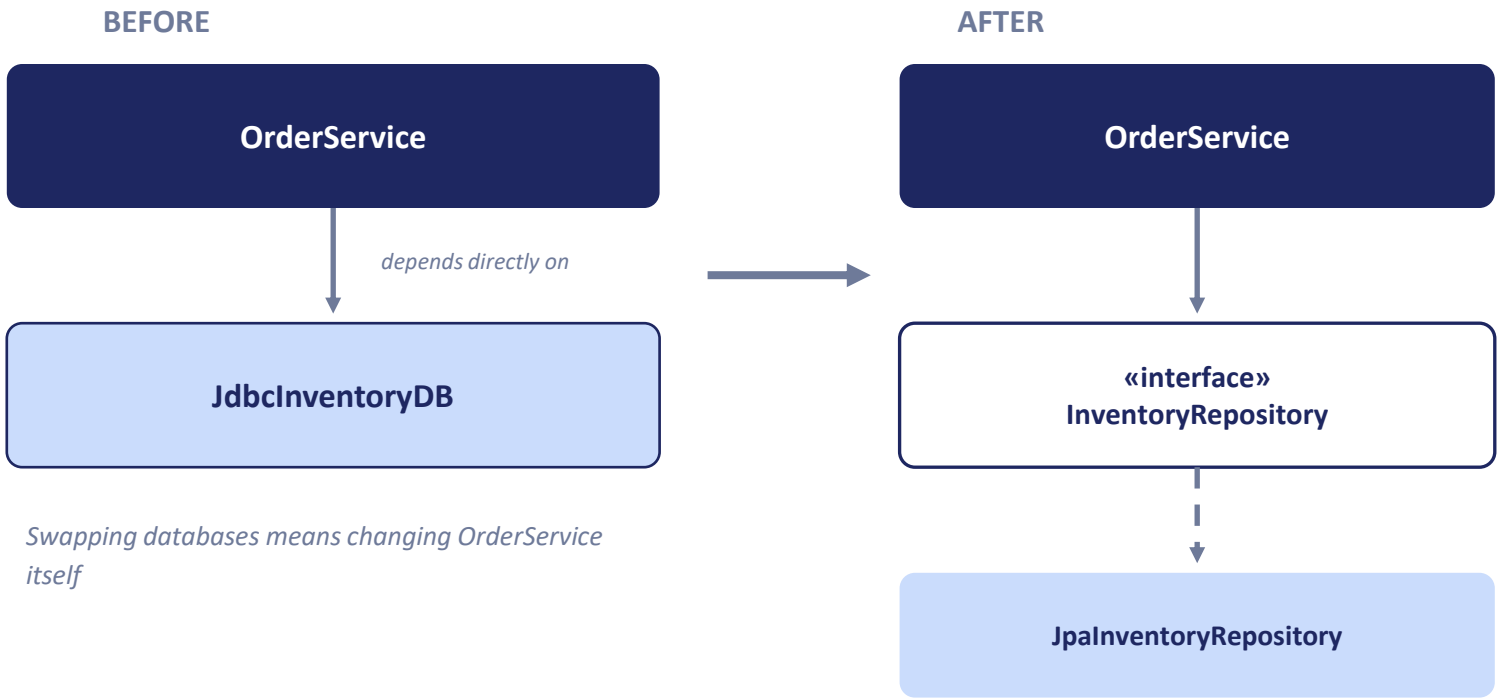
AccountingExporter

exportToAccountingSystem()

Each client depends only on the methods it actually uses

Dependency Inversion Principle (DIP)

Depend on abstractions, not concretions. High-level modules shouldn't depend on low-level modules — both should depend on interfaces.



In RetailOrderHub: OrderService and PaymentService depend on repository and gateway interfaces, not on Azure SQL or a specific payment provider directly.

Materials & Setup Checklist

What you'll need on hand before we start the labs

RetailOrderHub Codebase

The working Spring Boot project from Day 1, including the OrderManager class you analyzed in Lab 3

A GitHub Account

Your own free account, created live at the start of Lab 3 — see the GitHub Setup Guide, nothing to prepare beforehand

Facilitator Answer Keys

Full solution code for each SOLID step, available to check against if you get stuck

Hands-On Lab

Lab 1 • Participant Lab • ~35–40 min

SRP → DIP Refactor

Finish what the demo started — split OrderManager into OrderService, InventoryService, and PaymentService, apply DIP, and verify the app after every step.

What You'll Do

- Extract PaymentService (SRP) — same technique as the demo, but on your own
- Extract InventoryService, then OrderService — rename OrderManager and delete the dead validate methods
- Apply DIP: introduce InventoryRepository so InventoryService depends on an interface, not EntityManager
- Reflect (ISP): if OrderRepository grew invoice/accounting methods for other teams, how would you split the interface?

DURATION
~35–40 minutes

FORMAT
Individual • Verify each step

Hands-On Lab

Lab 2 • Participant Lab • ~20–25 min

OCP Exercise — Strategy Pattern

PaymentService still has an if/else chain from Lab 1. Prove you can add a new payment method by writing one new class and touching nothing else.

What You'll Do

- Create the PaymentStrategy interface and one strategy class per existing method (Credit Card, PayPal, Gift Card)
- Rewrite PaymentService to look up strategies from an injected Map — no if/else chain
- Prove it's Open-Closed: add ApplePayStrategy without opening PaymentService.java at all
- Reflect (LSP): is a GiftCardStrategy that silently returns true on insufficient balance an LSP violation?

DURATION
~20–25 minutes

FORMAT
Individual

Maintainability & Version Control

Writing Maintainable Code

Clean & modular

Small classes, clear names, one responsibility per unit — SOLID in practice

Testable

Depend on interfaces so tests can substitute fakes for PaymentStrategy or InventoryRepository

Documented

Class-level comments explain why, not just what — especially for non-obvious decisions

Version Control with Git

- Branch per change — never commit refactors directly to main
- Small, focused commits — one SOLID principle's worth of change per commit is a good rule of thumb
- Pull requests for self-review — via GitHub in today's lab
- History as documentation — a good commit message explains why, future-you will thank present-you

TAKE A BREAK

20–30 Minutes

Stretch, refill your coffee, and step away from the screen

Up next: your own GitHub Git workflow lab — branch, commit, push, PR, and merge

Hands-On Lab

Lab 3 • Participant Lab • ~25–30 min

GitHub Git Workflow

Set up your own GitHub repo, then practice the full branch → commit → push → PR → merge workflow by shipping a new DebitCardStrategy.

What You'll Do

- Create your GitHub account and an empty repo; push your Lab 1/Lab 2 code to main
- Branch, add DebitCardStrategy (without touching PaymentService.java), and commit
- Push the branch and open a pull request against main
- Self-review your own diff, leave a comment, then merge and sync your local main

DURATION
~25–30 minutes

FORMAT
Individual

Day 2 Wrap-Up

What we covered — and where it's going

Key Takeaways

- SOLID isn't academic — every principle traced directly to a Lab 3 finding
- OrderManager is now OrderService, InventoryService, and PaymentService
- Payment methods extend via new classes, not edited if/else chains
- Every dependency now points at an interface, not a concrete implementation

Expectation Coverage — Day 2

✓ **Implement design concepts (continued)**

Looking ahead — Day 3: We scale RetailOrderHub from a single server toward millions of users, add caching, and make it resilient — including the Circuit Breaker pattern.